

R4 — O display, e o custo da espera

Um protocolo gerado por software, medido no pino

Microcontroladores — DENE/UFMT

OBJETIVOS

- Justificar cada atraso do driver a partir dos mínimos da folha de dados, e não por cópia.
- Medir no osciloscópio o pulso de habilitação e o tempo de uma atualização de tela.
- Provocar, observar e explicar a falha intermitente causada por inicialização apressada.
- Comparar quatro bits contra oito bits **na mesma bancada**, e decidir com o número.

CH2-1 (LCD)	ON — é o único roteiro em que ele entra	NESTE KIT
CH5-3 e CH5-4	OFF, obrigatoriamente	
Chaves SWITCHS (PORTB)	Todas em OFF	

A verificar antes desta sessão, e é bloqueante: se as linhas de relé compartilham PORTD com o barramento do display, cada escrita de dado chaveia relé. CH5-3 e CH5-4 desligados é pré-requisito, não recomendação — conferir na serigrafia.

Também a verificar: se a linha R/W do módulo está aterrada. Se estiver, o indicador de ocupado é inacessível e toda a temporização por atraso fixo deste roteiro é consequência dessa escolha de hardware.

1 O primeiro periférico que não obedece na hora

Até o R3, escrever num pino produzia efeito imediato. O display não: ele tem controlador próprio, oscilador próprio, e leva tempo para executar o que recebe.

Operação	Tempo mínimo	Em ciclos de máquina
Comando comum ou caractere	~ 40 µs	160
Limpar o display	~ 1,6 ms	6 400
Energização inicial	~ 40 ms	160 000

Ignorar esses tempos produz o sintoma mais traiçoeiro do semestre: **o display funciona às vezes**.

ATENÇÃO

Um sistema que funciona quase sempre é mais perigoso que um que nunca funciona. O defeito não aparece na bancada — aparece em campo, sob condições que ninguém consegue reproduzir.

E há um efeito colateral visível: no XM118 os dados do display usam **o mesmo PORTD dos oito LEDs**. Escrever no display faz os LEDs piscarem.

NOTA

Isso é comportamento esperado, não defeito, e ilustra o que a §3 do R1 já dizia: pinos são recurso escasso, e o compartilhamento tem consequências visíveis.

2 Parte 1 — o driver, lido antes de usado

O professor fornece `lcd.c` e `lcd.h`. **Leia antes de gravar qualquer coisa.**

Tarefa 1. Localize `lcd_pulso()` e responda:

TAREFA

(a) Por que existem dois atrasos, um com o pino E alto e outro depois de baixá-lo?

(b) Cada um deles atende a qual mínimo da folha de dados do HD44780? Nomeie os dois.

(c) A folha de dados pede 230 ns de largura de pulso. `__delay_us(1)` produz 1000 ns. Por que não usar um valor menor, já que um único ciclo de máquina — 250 ns — já bastaria?

A resposta de (c) é a que separa copiar de projetar.

CONCEITO

Um ciclo de máquina atenderia quatro dos cinco mínimos. O `__delay_us(1)` é escolhido por **margem**: dá folga de mais de quatro vezes sobre o pior caso, o que absorve variação de módulo, de temperatura e de tensão sem que ninguém precise medir nada.

E há uma razão concreta: módulos alimentados a 2,7 V, e boa parte dos clones vendidos hoje, especificam 450 ns em vez de 230. Ainda cabe em 1000 ns; não caberia num pulso de 250 ns.

O ganho de “otimizar” o pulso seria de 750 ns por nibble. O preço seria um driver que funciona na sua bancada e falha na do colega, com um módulo de outro lote.

Tarefa 2. Com o osciloscópio no pino E, disparo na borda de subida, meça:

TAREFA

Grandeza	Previsto (P4)	Medido
Largura do pulso de habilitação		
Intervalo entre dois pulsos consecutivos		
Tempo entre os dois nibbles de um byte		

Compare com a previsão P4 da folha da aula 3.

3 Parte 2 — a inicialização, e a falha intermitente

Tarefa 3. Localize a sequência de inicialização e preencha:

TAREFA

0x30, três vezes	
0x20	
0x28	
0x0C	
0x06	
0x01	

Tarefa 4. Troque 0x0C por 0x0F e observe. O que mudou, e por quê?

TAREFA

Tarefa 5. Remova o `__delay_ms(50)` inicial e grave.

TAREFA

O display funciona? Funciona **sempre**? Ligue e desligue o kit três vezes:

Tentativa 1	
Tentativa 2	
Tentativa 3	

Este teste é o ponto central do roteiro, e o resultado costuma ser: funciona duas vezes e falha uma.

CONCEITO

Os 50 ms não são para o display: são para a **alimentação**. O controlador só garante o estado interno depois que a tensão estabiliza, e quem liga o kit pela USB não controla quanto tempo isso leva. Em alguns arranques a tensão sobe rápido o bastante; em outros, não.

Você não pode testar a ausência dessa falha. Só pode garantir a margem.

3.1 Os três 0x30

Tarefa 6. Remova **uma** das três repetições de 0x30. Grave, e ligue e desligue o kit cinco vezes. Depois pressione o reset — sem desligar — mais cinco vezes.

TAREFA

Os dois casos se comportam igual?

Não se comportam, e a diferença é o conteúdo da aula 3.

CONCEITO

Ao **energizar**, o controlador está sempre no mesmo estado: acabou de nascer, em oito bits. Duas repetições bastam, e a versão errada parece correta.

Ao **pressionar o reset**, o microcontrolador reinicia e o display não — ele continua em quatro bits, e pode estar esperando o nibble alto ou o nibble baixo. São três estados possíveis, e o terceiro só converge no terceiro pulso.

Testar apenas na energização esconde exatamente o defeito que a terceira repetição existe para evitar. É o mesmo padrão da Tarefa 5, com outro mecanismo.

4 Parte 3 — números no display

O display recebe **caracteres**, não números. Para mostrar 37 é preciso enviar '3' e depois '7'.

Tarefa 7. Escreva `lcd_numero(uint8_t v)` que mostre um valor de 0 a 255, e use para exibir um contador incrementando a cada 500 ms.

TAREFA

Antes de escrever: por que '0' + dígito funciona? O que garante que os dígitos sejam consecutivos na tabela de caracteres?

Tarefa 8. Deixe o contador passar de 100 para 99. O que aparece?

TAREFA

Compare com a previsão P1 da folha e explique. Depois corrija — **de duas formas diferentes**, uma na função de escrita e outra na formatação do número.

Este é o mesmo defeito da P1, com outro número: a segunda cadeia é mais curta que a primeira, e o display não apaga nada. Sobra o excedente do texto anterior.

NOTA

A correção que o resto do semestre vai usar é largura fixa — o número ocupa sempre as mesmas posições, completado com espaço à esquerda.

5 Parte 4 — quatro bits contra oito, medido

O kit liga as oito linhas de dados ao PORTD, então os dois modos são possíveis nesta bancada. A aula 3 argumentou que quatro bits economizam quatro pinos. Agora vocês medem o que isso custa.

Tarefa 9. Rode o mesmo texto nos dois modos e meça, com um pino auxiliar levantado antes e baixado depois da escrita:

TAREFA

	Oito bits	Quatro bits
Pulsos de E por byte		
Tempo de um caractere		
Tempo de uma linha de 16		
Pinos ocupados		

O resultado costuma surpreender: **quatro bits quase não custam tempo.**

CONCEITO

Cada byte gasta um pulso a mais, o que acrescenta poucos microssegundos — contra os 40 µs de execução do comando, que o controlador consome de qualquer jeito. A diferença fica na casa de 5%.

E economiza quatro pinos, que num projeto com sensor, aquecedor, ventoinha, teclado e comunicação são a diferença entre caber e não caber.

É por isso que quatro bits é o padrão, e não porque alguém queira escrever mais código. A decisão se justifica com os dois números medidos, e não com preferência.

6 Parte 5 — o custo da tela inteira

Tarefa 10. Meça o tempo de uma atualização completa: posicionar o cursor, escrever dezesseis caracteres, posicionar de novo, escrever mais dezesseis. TAREFA

Levante um pino auxiliar antes e baixe depois; meça a largura do pulso.

Tempo medido de uma atualização completa	
Em ciclos de máquina, a 250 ns	
Percentual do processador a cada 100 ms (P5)	
Percentual do processador a cada 500 ms (P5)	

Tarefa 11. Acrescente `lcd_comando(0x01)` — limpar — antes de cada atualização, e meça de novo. Depois olhe para a tela e descreva o que mudou visualmente. TAREFA

Os dois efeitos aparecem juntos: o tempo sobe alguns milhares de ciclos, e a tela **treme**, porque fica em branco por um instante a cada ciclo. CONCEITO

A prática correta é reposicionar o cursor e sobrescrever, com largura constante — que é a mesma correção da Tarefa 8, agora por um segundo motivo.

Guarde o número da Tarefa 10. Ele é o mais caro que este curso vai encontrar até o encontro 10, e é a unidade de comparação de tudo o que vem depois: uma conversão analógica custa sessenta ciclos, um tratamento de interrupção custa algumas dezenas, e uma atualização de tela custa alguns milhares. NOTA

7 Armadilhas frequentes

Sintoma	Causa provável
Display aceso, sem texto	Contraste, ou CH2-1 em OFF
Linha superior com quadrados escuros	Inicialização incompleta
Funciona às vezes	Falta de atraso após a energização
Caracteres embaralhados	Falta de espera após o comando de limpar
Texto na linha errada	Endereço de cursor: linha 1 começa em 0x40
Resíduo ao diminuir dígitos	Largura variável; não limpou a posição anterior
LEDs piscando junto	Esperado — PORTD é compartilhado
Relé chaveando a cada caractere	CH5-3 ou CH5-4 ligado. Desligue agora

8 Entrega

Entrega. As três tabelas de medida (Tarefas 2, 9 e 10), o registro das três tentativas da Tarefa 5, e o resultado da Tarefa 6 nos dois casos — energização e reset. TAREFA

Responda também:

(a) Por que `__delay_us(1)` e não um único ciclo de máquina? Cite o mínimo que justifica a escolha e a margem obtida.

(b) Na Tarefa 6, por que o defeito aparece no reset e não na energização?

(c) Com os números da Tarefa 9, você escolheria quatro ou oito bits para um produto? Justifique com as duas grandezas.

(d) A previsão P2 dizia respeito a trocar LATD por PORTD no driver. Você testou? O que aconteceu, e por quê?

No R5: a entrada e o conversor. Vocês vão ler um botão que não responde, e descobrir por quê — a resposta está no que existe atrás do pino, e não no código.

NOTA

E o LM35 entra: temperatura na tela, com a conta que vocês trouxeram da última sessão.